COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	09/10/2026
NAME:	Uchi, Mao Sophie S. Verano, James Daniel M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-On Act 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

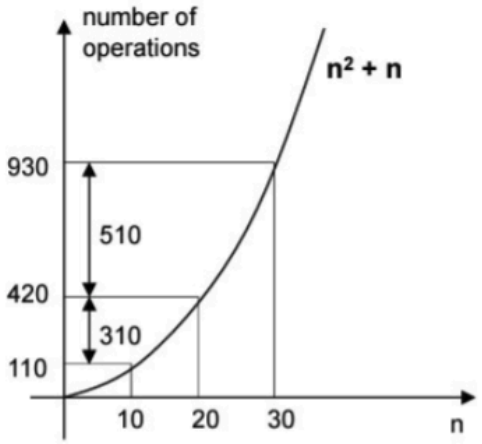
- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

Answer:	Total Comparisons = O(M * N) = **$O(N^2)$** Worst Case Time Complexity

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

The Analysis the we've come up with the given graph representation, the graph represents the quadratic relationship worse time complexity case **O(n^2)** between input **size (n)** (represented in the x-axis) and the **total number of operations** (represented in the y-axis). Given to the table beside the graph, as the array size increases by 10 or to be exact, doubles, the number of needed operations quadruples as it iterates more using the worst case time complexity of the algorithm. As represented, a size of 10 needs an amount of 110 operations to be able to iterate each of the arrays inside the array list. The moment we increased the array size, the quadratic increased the needed operations for the program to finish all of the arrays in the array list.

n = 30 = 420 (operations previously) + 310 (quadruples) = 930.

With the growth of the quadratic time complexity, it entirely indicates that the diff algorithm becomes too hassle for large datasets. In terms of space, it is too big of a gap and is inefficient for large datasets.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Code

```
#include <iostream>
#include <vector>
```

```cpp
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student {
    private:
        std::pair<int, int> name;
        bool vaccinated;

    public:
    Student(std::pair<int , int> n, bool v) : name(n), vaccinated(v) {
    }

    auto get_name() {return name;}
    auto is_vaccinated() {return vaccinated;}

    // Two people are same if they have the same name
    bool operator == (const Student& p) const{
        return this-> name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator < (const Student& p) const{
         return this->name < p.name;
    }

    bool operator > (const Student& p) const{
         return this->name > p.name;
    }
};

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
```

```
    //the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type>
uniform_dist(1, max);

    //generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand),
uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);
        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
            else if (mid_element == P && mid_element.is_vaccinated()
== true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);
        // Student not found in the sequence and therefore should be
vaccinated
        if (range_length == 1)
            return true;
    }
}
```

```
void search_test(int size, Student p) {
    std::vector<Student> people;

    //Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    //measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);

    //stop the clock
    std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
    std::cout << "Time taken to search = "
    << std::chrono::duration_cast<std::chrono::microseconds> (end -
begin).count()
    << " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
        << ") " << "needs vaccination." << std::endl;
    }
    else {
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
        << ") " << "does not need vaccination." << std::endl;
    }
}
```

```
int main() {
    // generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Uchi's Answer:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	0 microsecond	`Time taken to search = 0 microseconds` `Student (988 307) needs vaccination.`	This algorithm implies a binary search with a logarithmic time complexity as the input size this small would run faster with O(logn), also checking on how long the search took, it is less than a microsecond.
10,000	1 microsecond	`Time taken to search = 1 microseconds` `Student (988 307) needs vaccination.`	With a scale increase tenfold between the first and second input sizes proves further that the time complexity of this algorithm is logarithmic. From the execution time, an additional microsecond increased.
100,000	2 microseconds	`Time taken to search = 2 microseconds` `Student (988 307) needs vaccination.`	With a scale increase tenfold between the first, second and third input sizes, the execution time of the code elapsed quicker than the timer resolution. This further demonstrates the effectiveness of using

			O(log n) to this search that the timer can't detect how fast the execution time taken.

Verano's Answer:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	0 microsecond	`Time taken to search = 0 microseconds` `Student (589 361) needs vaccination.`	The searching algorithm with the given execution speed is almost instantaneous as it represents a logarithmic time complexity of O(log n), which is known for binary searches.
10,000	1 microsecond	`Time taken to search = 1 microseconds` `Student (589 361) needs vaccination.`	Comparing the execution speed in the array that has a size of 1,000, when we increased the dataset size by tenfold, we can see that the execution speed increased by a total of 1 microseconds. Based on the definition of O(log n), it increases exponentially, which this applies to the scalability of O(log n) time complexity.
100,000	2 microseconds	`Time taken to search = 2 microseconds` `Student (589 361) needs vaccination.`	Comparing this array size to the previous two, with 100,000 elements, the execution speed only took 2 microseconds. Given how minimal the increase in its execution time is, the only time complexity that directly aligns with this kind of growth is the O(log n) as

			its exponential growth behavior.

Analysis

In the provided table above, as we applied the experimental analysis, as you can see that there's not much of a difference as it both follows the same code structure but **highly depends on the computer's performance**. After all, it both uses the same time complexity **O(log n)** growth, which is exponentially. It is expected that without too much of a difference in computer components such as Higher CPU clock speed, or large and faster CPU cache, The difference in the execution speed won't be too much of a gap. It all comes down to the hardware specifications that each computer has to process the execution speed. In reality, there is still indeed a small difference in each execution speed, it is just super unnoticeable given how small its differences.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

```cpp
#include <iostream>
#include <vector>
#include <chrono>

bool Unique(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if(A[i] == A[j]) return false;
```

```
        }
    }
    return true;
}

int main() {
    std::cout << "=> Worst Time Complexity: \n" << std::endl;
    std::vector<int> sizes = {1000, 10000, 100000};
    for (int n : sizes) {
        std::vector<int> A(n);
        for (int i = 0; i < n; i++) A[i] = i;

        auto start = std::chrono::high_resolution_clock::now();
        Unique(A);
        auto end = std::chrono:: high_resolution_clock::now();

        auto duration =
std::chrono::duration_cast<std::chrono::microseconds>(end -
start).count();
        std::cout << "N = " << n << " | Time: " << duration << "
microseconds\n";

    }

    std::cout << "\n=> Best Time Complexity: \n" << std::endl;
    for (int n : sizes) {
        std::vector<int> A(n);
        for ( int i = 0; i < n; i++) {
            A[i] = i + 1;
        }
        A[1] = A[0];


        auto start = std::chrono::high_resolution_clock::now();
        Unique(A);
        auto end = std::chrono:: high_resolution_clock::now();
```

```
        auto duration =
std::chrono::duration_cast<std::chrono::microseconds>(end -
start).count();
        std::cout << "N = " << n << " | Time: " << duration << "
microseconds\n";
    }
    return 0;
}
```

Uchi's Answer:

Theoretical Analysis

With a unique array implemented, we can compare each element whether they're similar or not. For index i and index j are equal, they must return false as both elements are not unique. For index i and index j are not equal, they must return true as both elements are unique. The runtime of this algorithm, the best case would be O(1), as comparing the first index and second index as not unique would stop the algorithm. The worst case would be $O(n^2)$, as the algorithm will compare every non-duplicate element until an equal element is found.

Cases	No. of Comparisons	Time Complexity
Best	1	O(1)
Worst	$\frac{(n^2-n)}{2}$	$O(n^2)$

(combination)

n = N	
How many times is operation 1 executed?	1
How many times is operation 4 executed?	1
How many times is operation 8 and 9 executed in total?	1
Total so far:	**3**

	Best Case: The elements in an array are NOT unique
	How many times is operation 2 executed? — n-1
	How many times is operation 3 executed? — N
	Total number of operations: — **2n+2**

	Worst Case: The elements in an array are unique
	How many times is operation 2 executed? — n+1
	How many times is operation 3 executed? — N
	How many times is operation 5 executed? — n+1
	How many times is operation 6 executed? — N
	How many times is operation 7 executed? — N
	Total number of operations: — **5n+5**

Experimental Analysis

Best Case: The elements in an array are NOT unique

```
for (int i = 0; i < size; i++)
    A[i] = i + 1;
A[1] = A[0];
```

Input Size (N)	Execution Speed	Screenshot
1,000	0 microsecond	`N = 1000 | Time: 0 microseconds`
10,000	0 microsecond	`N = 10000 | Time: 0 microseconds`

100,000	0 microsecond

```
N = 100000 | Time: 0 microseconds
```

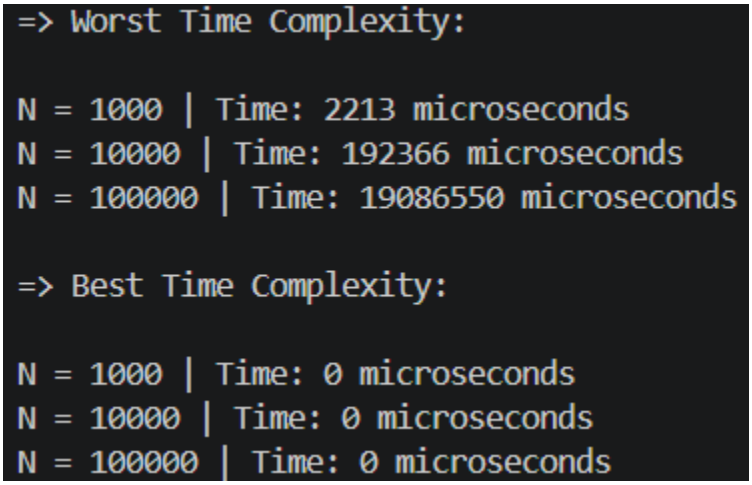
Worst Case: The elements in an array are unique

```
for (int i = 0; i < size; i++) {
    A[i] = i + 1;
}
A[1] = A[0];
```

Input Size (N)	Execution Speed	Screenshot
1,000	1,328 microseconds	`N = 1000 | Time: 1328 microseconds`
10,000	133,012 microseconds	`N = 10000 | Time: 133012 microseconds`
100,000	13,455,519 microseconds	`N = 100000 | Time: 13455519 microseconds`

Analysis and Comparison	From both of the analyses above, I can observe that the input size 1,000 has the fastest execution time as not only it is one of the smallest from the three input sizes, its execution speed can not be detected by the clock precision. The best case for this algorithm is still O(1) no matter how large the input size, as long as the two elements are equal from the first comparison. The worst case for this algorithm would be $O(n^2)$ as the input size doubles, running time will increase forthfold, producing this algorithm with a growth rate of quadratic.

Verano's Answer:	
Theoretical Analysis	As represented in the code converted algorithm above, the bool Unique function handles a time complexity of O(**n^2**) as each loop is represented by **n** times. As per iteration, in the unique function, it represents **n(n + 1)** which is why the worst time complexity is **O(n^2)**. On the other hand, it can also achieve a best-case time complexity of **O(1)** if the two elements are identical **(A[0] == A[1])**.

Experimental Analysis	 In the experimental analysis, when we executed the program consisting of 3 large gap sizes to identify the difference of each on how long would the time complexity take to execute and iterate all of the elements. As represented by the output above, in the worst time complexity, **as the number of sizes in an array increases**, the lo**nger it takes to fully finish the execution of the iteration**. The difference in each gap increases depending on how many were increased in each array. Comparing n = 1,000 to n = 100,000, almost **ten-thousandfold** would need to be multiplied to the time complexity of the n = 1,000 to have the similar microseconds of n = 100,000. While, in the **Best Time Complexity**, given that if **n = 1**, no matter the size the array has, if the value of n is equivalent to 1, it will fully be able to meet the condition and end the program's iteration.
Analysis and Comparison	Overall, In the theoretical approach, following the worst-time complexity **O(n^2)**, when N increases depending on how much is being increased, (e.g. 10,000 to 20,000 means n is doubled) the runtime increases approximately to 2 more. Given to the experiment, 1,000 and 10,000, the runtime **approximately is multiplied thousandfold**. This result confirms the theoretical time complexity of **O(n^2)** quadratic prediction.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>

int rpower(int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

int brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        int y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    int x = 2;
    int exponents[] = {10000, 20000, 30000};

    std::cout << std::left << std::setw(12) << "Exponent (N) "
```

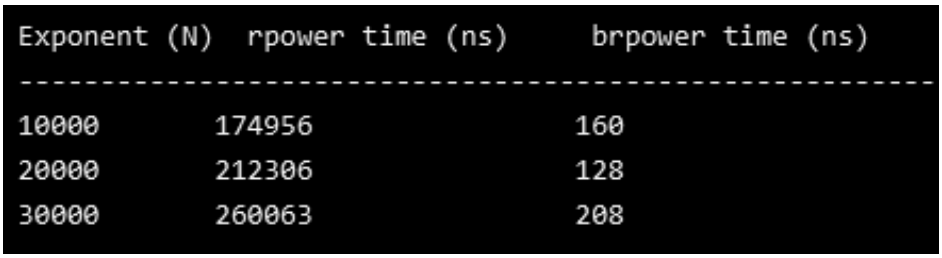
```
              << std::setw(22) << " rpower time (ns)"
              << std::setw(22) << "brpower time (ns)" << std::endl;
    std::cout <<
"-------------------------------------------------------" <<
std::endl;

    for (int n : exponents) {
        // rpower measurement
        auto start1 = std::chrono::high_resolution_clock::now();
        rpower(x, n);
        auto end1 = std::chrono::high_resolution_clock::now();
        auto dur1 =
std::chrono::duration_cast<std::chrono::nanoseconds>(end1 -
start1).count();

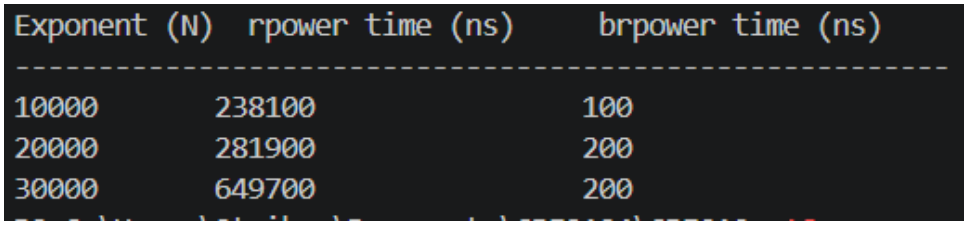
        // brpower measurement
        auto start2 = std::chrono::high_resolution_clock::now();
        brpower(x, n);
        auto end2 = std::chrono::high_resolution_clock::now();
        auto dur2 =
std::chrono::duration_cast<std::chrono::nanoseconds>(end2 -
start2).count();
        std::cout << std::left << std::setw(12) << n
        << std::setw(22) << dur1
        << std::setw(22) << dur2 << std::endl;
    }

    return 0;
}
```

Uchi's Answer:	
Theoretical Analysis	From these two power calculating algorithms, the algorithm rPower works by every recursive step, it should decrement the exponent n by 1, until the input size becomes zero.

	The base case: `if (n == 0) return 1;` is a conditional function that stops the recursion case: `return x * rpower(x, n - 1);` to prevent infinite loops that can cause the program to crash. The time complexity of this function is O(n), for both best case and worst case, from the recursive case, it is obvious as every recursive step will decrement the exponent n to one, which is a linear approach of time complexity. The algorithm brPower works by using the divide and conquer paradigm, each step, you'll be dividing the nth exponent by 2. Similar to rPower, the base case also returns 1 if the nth exponent is equal to 0, but this is where the power calculation differs from the two, which is the recursive case. From the algorithm, after the base case line, there are two recursive cases, first is the Odd recursive case, (n=odd), where the nth exponent is odd and must be incremented by one to divide the n by 2. Second is the Even recursive case, (n=even), where the nth exponent is even and must divide the n by 2. The time complexity of this function is O(log n), for both best case and worst case, from the recursive case it is also obvious that every recursive step, you'll be dividing the nth exponent to 2, which is a logarithmic approach of time complexity.
Experimental Analysis	 From the table above, evidently, brpower has a much more efficient runtime than rpower, with an approximate difference of nanoseconds, brpower is lesser by a thousandfold than rpower. The time complexity also contributes to the runtime of these power calculations, hence why the rPower is slower than the brPower, as rPower's time complexity is linear, while the brPower's time complexity is logarithmic. As the exponent increases, both its power calculation also increases, but the time complexity still works similarly, as rPower works linearly, its runtime is much more substantial compared to brPower's logarithmic process.
Analysis and Comparison	Based on the two analyses, rPower is developed to approach the exponent and decrease it one by one after every recursive step until the exponent reaches zero, while brPower is developed to approach the exponent and apply the divide and conquer method and decrease it with a divisor of 2. From the provided table from the Experimental Analysis, brPower's performance compared to rPower is much more efficient and better overall due to brPower's logarithmic time complexity.

Verano's Answer:

Theoretical Analysis	In the theoretical analysis, there are several time complexity that each function has, for such the **rpower function**, consisting of only a linear recursion, which only holds **(n-1)**, which is a time complexity of **O(n)**. proceeding to the next function, which is the **brpower function**, which is a logarithmic recursion, which can be represent the behavior of a **O(log n)** time complexity. Therefore, The program's Worst-case time complexity is **O(log n)** given that it increases the number of operations needed to finish the execution of the iteration. On the other hand, same with the problem's 1 theoretical analysis, It has the best-case time complexity **O(1)**. it is a value where n = 0 or 1 without following indexing property. For the reason that, before multiple recursion and multiple operation happens, the condition has already been met, therefore ending the execution as its fastest time.
Experimental Analysis	 Comparing the Theoretical Analysis to the experimental analysis, As you can see, as the exponent increases, the **rpower** and **brpower time** also increases. But as per usual, this may differ since network is a possible factor that can hindrance the time complexity of each comparison. The execution time grows drastically, because it reflects its linear time complexity given to how much is needed to be incremented in the exponent size that requires N sequential recursive calls. In the **brpower**, this evidently supports that, it is entirely still dependent on the network or performance of your computer. As 20,000 and 30,000 got the same time complexity, this means that they both finished the execution at **200ns**.
Analysis and Comparison	Comparing the time complexity behavior of the 2 functions **rpower** & **brpower**: In the **rpower function**, it uses a linear recursive call, only operating at (n + 1) time complexity at most times. Requiring n multiplication operations only representing the theoretical time complexity of **O(n)**. While in the **brpower function**, It employs a divide-and-conquer exponentiation. It halves the exponent at each step, (n = n/2 -> n/4) until it reduces to the total time complexity of log2(n) + 1, which represents the theoretical time complexity of **O(log n).**

Uchi's Conclusion
My conclusion for this activity is that not only do we get to learn on how to create a program using C++, we also get to analyze algorithms on how it accesses and organizes our data. In learning how to identify the run time and

space utilization of an algorithm, are the better ways of understanding how much operations, comparisons and time complexity are indicated in your algorithm. It is within reason to be knowledgeable about this information as each algorithm has different growth rate functions to analyze which functions are better for this algorithm, hence the best and worst case for each algorithm is important. It is also important having a faster runtime on your program as your computer hardware specs, such as your cpu, will determine how quick your program will execute.

Verano's Conclusion

In accomplishing this hands-on activity, I was able to unravel and explore more deeper into the algorithm's time and space complexity. With the different procedural and supplementary activity, I was able to further understand and enhance my **critical thinking skills** in identifying **worst and best time complexity** as well as their **space auxiliary**. Doing the theoretical and experimental analysis, I learned new things and areas for future improvement that **includes memory stack consumption** and **allocation** alongside **execution time**. One of the polished things that I also was able to further realize is that, in the theoretical analysis, it strongly validates the complexity models. As the experimental being the represented value, it served as the evidential that the theoretical analysis is closely correlated per own analysis of the time complexity. With also some knowledge from the previous topic of DSA, there were topics that were recalled in the supplementary activity which is the **divide-and-conquer** type of algorithm, and how linear recursion becomes inefficient and memory-heavy for large inputs, compared to binary exponentiation representing the **O(log n)** optimizes both time and space auxiliary.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance, Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **How to print out the rpower and brpowers, and a way to represent the recursion process in problem 2.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.